

1.University Student Management System

- Implement classes for `Student`, `Course`, and `Department` using inheritance and encapsulation.
- Create a package `com.university.management`.
- Define interfaces for `ExamEligibility` and `ScholarshipEligibility`.
- Build logic to calculate GPA and eligibility based on input data.
- Incorporate **exception handling** to manage: Invalid input data (e.g., negative GPA or grades out of range).

(Use constructors, static, this, and final keywords wherever applicable)

Code :

```
package com.university.management;
```

```
import java.util.Scanner;
```

```
interface ExamEligibility {  
    boolean isEligibleForExam();  
}
```

```
interface ScholarshipEligibility {  
    boolean isEligibleForScholarship();  
}
```

```
class Department {  
    protected String departmentName;  
  
    public Department(String departmentName) {  
        this.departmentName = departmentName;  
    }  
  
    public void displayDepartment() {  
        System.out.println("Department: " + departmentName);  
    }  
}
```

```
}
```

```
class Course extends Department {
```

```
    protected String courseName;
```

```
    protected int courseCode;
```

```
    public Course(String departmentName, String courseName, int courseCode) {
```

```
        super(departmentName);
```

```
        this.courseName = courseName;
```

```
        this.courseCode = courseCode;
```

```
    }
```

```
    public void displayCourse() {
```

```
        System.out.println("Course Name: " + courseName);
```

```
        System.out.println("Course Code: " + courseCode);
```

```
    }
```

```
}
```

```
class Student extends Course implements ExamEligibility, ScholarshipEligibility {
```

```
    private String studentName;
```

```
    private int rollNo;
```

```
    private double marks[];
```

```
    private double gpa;
```

```
    static int studentCount = 0;
```

```
    final String universityName = "MIT AOE";
```

```
public Student(String departmentName,
               String courseName,
               int courseCode,
               String studentName,
               int rollNo,
               double marks[]) {

    super(departmentName, courseName, courseCode);

    this.studentName = studentName;
    this.rollNo = rollNo;
    this.marks = marks;

    studentCount++;

    calculateGPA();
}

public void calculateGPA() {

    double sum = 0;

    for(double mark : marks) {

        if(mark < 0 || mark > 100) {
            throw new IllegalArgumentException(
                "Marks should be between 0 and 100"
            );
        }
    }
}
```

```
        sum += mark;
    }

    double average = sum / marks.length;

    this.gpa = average / 20.0;

    if(gpa < 0 || gpa > 10) {
        throw new IllegalArgumentException(
            "Invalid GPA calculated"
        );
    }
}

@Override
public boolean isEligibleForExam() {
    return gpa >= 4.0;
}

@Override
public boolean isEligibleForScholarship() {
    return gpa >= 8.0;
}

public void displayStudent() {

    System.out.println("\n===== STUDENT DETAILS =====");
```

```
System.out.println("University: " + universityName);
```

```
displayDepartment();
```

```
displayCourse();
```

```
System.out.println("Student Name: " + studentName);
```

```
System.out.println("Roll No: " + rollNo);
```

```
System.out.println("GPA: " + gpa);
```

```
System.out.println("Exam Eligible: " +  
    (isEligibleForExam() ? "YES" : "NO"));
```

```
System.out.println("Scholarship Eligible: " +  
    (isEligibleForScholarship() ? "YES" : "NO"));
```

```
System.out.println("Total Students: " + studentCount);
```

```
}
```

```
}
```

```
public class UniversityManagementSystem {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        try {
```

```
            String dept = sc.nextLine();
```

```
String course = sc.nextLine();
```

```
int code = sc.nextInt();
```

```
sc.nextLine();
```

```
String name = sc.nextLine();
```

```
int roll = sc.nextInt();
```

```
double marks[] = new double[5];
```

```
for(int i = 0; i < 5; i++) {
```

```
    marks[i] = sc.nextDouble();
```

```
    if(marks[i] < 0 || marks[i] > 100) {
```

```
        throw new IllegalArgumentException(
```

```
            "Marks must be between 0 and 100"
```

```
        );
```

```
    }
```

```
}
```

```
Student s1 = new Student(
```

```
    dept,
```

```
    course,
```

```
    code,
```

```
    name,
```

```
    roll,
```

```

        marks

    );

    s1.displayStudent();

}

catch(IllegalArgumentException e) {

    System.out.println("Exception: " + e.getMessage());
}

catch(Exception e) {

    System.out.println("Unexpected Error: " + e.getMessage());
}

finally {

    System.out.println("\nProgram Executed Successfully.");
}

sc.close();
}
}

```

2. Online Shopping System

- Use OOP to define `Product`, `Cart`, and `Order` classes.
- Create a package `com.shopping.cart` and interface `Discountable`.
- Apply discount logic, calculate total bill, and simulate cart checkout.
- Integrate robust **exception handling** to manage:

Invalid operations such as adding products with **negative quantity or price**.

(Use constructors, static, this, and final keywords wherever applicable)

Code :

```
package com.shopping.cart;
```

```
import java.util.ArrayList;
```

```
import java.util.Scanner;
```

```
interface Discountable {  
    double applyDiscount();  
}
```

```
class Product implements Discountable {
```

```
    private String productName;
```

```
    private double price;
```

```
    private int quantity;
```

```
    static int productCount = 0;
```

```
    final String storeName = "Online Shopping Store";
```

```
    public Product(String productName, double price, int quantity) {
```

```
        if(price < 0) {  
            throw new IllegalArgumentException(  
                "Price cannot be negative"  
            );  
        }  
    }
```

```
        if(quantity < 0) {
```

```
        throw new IllegalArgumentException(  
            "Quantity cannot be negative"  
        );  
    }
```

```
    this.productName = productName;  
    this.price = price;  
    this.quantity = quantity;
```

```
    productCount++;  
}
```

```
public String getProductName() {  
    return productName;  
}
```

```
public double getPrice() {  
    return price;  
}
```

```
public int getQuantity() {  
    return quantity;  
}
```

```
public double getTotalPrice() {  
    return price * quantity;  
}
```

```
@Override
```

```
public double applyDiscount() {
```

```

double total = getTotalPrice();

if(total >= 5000) {
    return total * 0.10;
}

return total * 0.05;
}

public void displayProduct() {

    System.out.println("Product Name: " + productName);
    System.out.println("Price: " + price);
    System.out.println("Quantity: " + quantity);
    System.out.println("Total Price: " + getTotalPrice());
}
}

class Cart {

    private ArrayList<Product> products;

    public Cart() {
        products = new ArrayList<>();
    }

    public void addProduct(Product p) {
        products.add(p);
    }

    public double calculateBill() {

```

```
double total = 0;
```

```
for(Product p : products) {
```

```
    double productTotal = p.getTotalPrice();
```

```
    double discount = p.applyDiscount();
```

```
    total += (productTotal - discount);
```

```
}
```

```
return total;
```

```
}
```

```
public void displayCart() {
```

```
    System.out.println("\n===== CART DETAILS =====");
```

```
    for(Product p : products) {
```

```
        p.displayProduct();
```

```
        System.out.println("-----");
```

```
    }
```

```
    System.out.println("Final Bill: " + calculateBill());
```

```
}
```

```
}
```

```
class Order {
```

```
    private Cart cart;
```

```

public Order(Cart cart) {
    this.cart = cart;
}

public void checkout() {

    System.out.println("\n===== ORDER CHECKOUT =====");

    cart.displayCart();

    System.out.println("Order Placed Successfully");
}
}

public class OnlineShoppingSystem {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        try {

            Cart cart = new Cart();

            int n = sc.nextInt();

            sc.nextLine();

            for(int i = 0; i < n; i++) {

                String name = sc.nextLine();

```

```
double price = sc.nextDouble();
```

```
int quantity = sc.nextInt();
```

```
sc.nextLine();
```

```
Product p = new Product(name, price, quantity);
```

```
cart.addProduct(p);
```

```
}
```

```
Order order = new Order(cart);
```

```
order.checkout();
```

```
System.out.println("Total Products: " +
```

```
    Product.productCount);
```

```
}
```

```
catch(IllegalArgumentException e) {
```

```
    System.out.println("Exception: " + e.getMessage());
```

```
}
```

```
catch(Exception e) {
```

```
    System.out.println("Unexpected Error: " + e.getMessage());
```

```
}
```

```
finally {
```

```

        System.out.println("\nProgram Executed Successfully");
    }

    sc.close();
}
}

```

3.Railway Reservation System

- Create classes for `Train`, `Passenger`, `Ticket`. Use inheritance and constructors.
- Create a package `com.railway.reservation`.
- Use interface `Reservable`.
- Write logic for seat booking, cancellation, and fare calculation.
- Include comprehensive **exception handling** to manage:
 - **Overbooking** or booking when seats are full.
 - **Invalid passenger data** (e.g., missing fields or invalid age).

(Use constructors, static, this, and final keywords wherever applicable)

Code :

```
package com.railway.reservation;
```

```
import java.util.Scanner;
```

```
interface Reservable {
    void bookSeat();
    void cancelSeat();
}

```

```
class Train {
```

```
protected int trainNumber;  
protected String trainName;  
protected int totalSeats;  
protected int availableSeats;
```

```
static int totalBookings = 0;
```

```
final String railwayName = "Indian Railways";
```

```
public Train(int trainNumber,  
             String trainName,  
             int totalSeats) {
```

```
    if(totalSeats <= 0) {  
        throw new IllegalArgumentException(  
            "Total seats must be greater than 0"  
        );  
    }
```

```
    this.trainNumber = trainNumber;  
    this.trainName = trainName;  
    this.totalSeats = totalSeats;  
    this.availableSeats = totalSeats;  
}
```

```
public void displayTrain() {
```

```
    System.out.println("Railway Name: " + railwayName);  
    System.out.println("Train Number: " + trainNumber);
```

```
        System.out.println("Train Name: " + trainName);
        System.out.println("Available Seats: " + availableSeats);
    }
}
```

```
class Passenger extends Train {
```

```
    protected String passengerName;
```

```
    protected int age;
```

```
    public Passenger(int trainNumber,
```

```
        String trainName,
```

```
        int totalSeats,
```

```
        String passengerName,
```

```
        int age) {
```

```
        super(trainNumber, trainName, totalSeats);
```

```
        if(passengerName == null ||
```

```
            passengerName.trim().isEmpty()) {
```

```
            throw new IllegalArgumentException(
```

```
                "Passenger name cannot be empty"
```

```
            );
```

```
        }
```

```
        if(age <= 0 || age > 120) {
```

```
            throw new IllegalArgumentException(
```

```

        "Invalid passenger age"

    );
}

this.passengerName = passengerName;
this.age = age;
}

public void displayPassenger() {

    System.out.println("Passenger Name: " + passengerName);
    System.out.println("Passenger Age: " + age);
}
}

```

```

class Ticket extends Passenger implements Reservable {

```

```

    private int seatsBooked;
    private double fare;

```

```

    public Ticket(int trainNumber,
                  String trainName,
                  int totalSeats,
                  String passengerName,
                  int age,
                  int seatsBooked) {

```

```

        super(trainNumber,
              trainName,

```

```

        totalSeats,
        passengerName,
        age);

    if(seatsBooked <= 0) {

        throw new IllegalArgumentException(
            "Seats booked must be greater than 0"
        );
    }

    this.seatsBooked = seatsBooked;

    calculateFare();
}

public void calculateFare() {

    fare = seatsBooked * 500;
}

@Override
public void bookSeat() {

    if(seatsBooked > availableSeats) {

        throw new IllegalArgumentException(
            "Seats are full. Overbooking not allowed"
        );
    }
}

```

```
}
```

```
availableSeats -= seatsBooked;
```

```
totalBookings++;
```

```
System.out.println("Seat Booked Successfully");
```

```
}
```

```
@Override
```

```
public void cancelSeat() {
```

```
    availableSeats += seatsBooked;
```

```
    totalBookings--;
```

```
    System.out.println("Ticket Cancelled Successfully");
```

```
}
```

```
public void displayTicket() {
```

```
    System.out.println("\n===== TICKET DETAILS =====");
```

```
    displayTrain();
```

```
    displayPassenger();
```

```
    System.out.println("Seats Booked: " + seatsBooked);
```

```
    System.out.println("Fare: " + fare);
```

```
        System.out.println("Total Bookings: " + totalBookings);
    }
}
```

```
public class RailwayReservationSystem {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        try {
```

```
            int trainNumber = sc.nextInt();
```

```
            sc.nextLine();
```

```
            String trainName = sc.nextLine();
```

```
            int totalSeats = sc.nextInt();
```

```
            sc.nextLine();
```

```
            String passengerName = sc.nextLine();
```

```
            int age = sc.nextInt();
```

```
            int seatsBooked = sc.nextInt();
```

```
            Ticket t1 = new Ticket(
```

```
                trainNumber,
```

```
                trainName,
```

```
        totalSeats,
        passengerName,
        age,
        seatsBooked
    );

    t1.bookSeat();

    t1.displayTicket();

}

catch(IllegalArgumentException e) {

    System.out.println("Exception: " + e.getMessage());
}

catch(Exception e) {

    System.out.println("Unexpected Error: " + e.getMessage());
}

finally {

    System.out.println("\nProgram Executed Successfully");
}

sc.close();
}
```

```
}
```

4. Employee Payroll Management System

- Define `Employee` as base class with subclasses like `Manager`, `Clerk`.
- Use package `com.company.hr` and interface `Payable`.
- Implement salary calculation based on working days and incentives.
 - Handle **invalid salary inputs** or **negative working days**.

(Use constructors, static, this, and final keywords wherever applicable)

Code :

```
package com.company.hr;
```

```
import java.util.Scanner;
```

```
interface Payable {  
    double calculateSalary();  
}
```

```
class Employee {  
  
    protected int empId;  
    protected String empName;  
    protected double basicSalary;  
  
    static int employeeCount = 0;  
  
    final String companyName = "ABC Company";  
  
    public Employee(int empId,  
                    String empName,
```

```
        double basicSalary) {

    if(empName == null || empName.trim().isEmpty()) {

        throw new IllegalArgumentException(

            "Employee name cannot be empty"

        );
    }

    if(basicSalary < 0) {

        throw new IllegalArgumentException(

            "Salary cannot be negative"

        );
    }

    this.empld = empld;
    this.empName = empName;
    this.basicSalary = basicSalary;

    employeeCount++;
}

public void displayEmployee() {

    System.out.println("Company Name: " + companyName);
    System.out.println("Employee ID: " + empld);
    System.out.println("Employee Name: " + empName);
    System.out.println("Basic Salary: " + basicSalary);
}
```

```
}  
}
```

```
class Manager extends Employee implements Payable {
```

```
    private int workingDays;
```

```
    private double incentive;
```

```
    public Manager(int empld,  
                   String empName,  
                   double basicSalary,  
                   int workingDays,  
                   double incentive) {
```

```
        super(empld, empName, basicSalary);
```

```
        if(workingDays < 0) {
```

```
            throw new IllegalArgumentException(  
                "Working days cannot be negative"  
            );  
        }
```

```
        if(incentive < 0) {
```

```
            throw new IllegalArgumentException(  
                "Incentive cannot be negative"  
            );  
        }
```

```
    this.workingDays = workingDays;
    this.incentive = incentive;
}
```

```
@Override
```

```
public double calculateSalary() {
```

```
    return (basicSalary / 30) *
        workingDays + incentive;
}
```

```
public void displayManager() {
```

```
    System.out.println("\n===== MANAGER DETAILS =====");
```

```
    displayEmployee();
```

```
    System.out.println("Working Days: " + workingDays);
```

```
    System.out.println("Incentive: " + incentive);
```

```
    System.out.println("Total Salary: " +
```

```
        calculateSalary());
```

```
}
```

```
}
```

```
class Clerk extends Employee implements Payable {
```

```
    private int workingDays;
```

```
public Clerk(int empId,
             String empName,
             double basicSalary,
             int workingDays) {

    super(empId, empName, basicSalary);

    if(workingDays < 0) {

        throw new IllegalArgumentException(
            "Working days cannot be negative"
        );
    }

    this.workingDays = workingDays;
}

@Override
public double calculateSalary() {

    return (basicSalary / 30) *
        workingDays;
}

public void displayClerk() {

    System.out.println("\n===== CLERK DETAILS =====");
}
```

```
displayEmployee();
```

```
System.out.println("Working Days: " + workingDays);
```

```
System.out.println("Total Salary: " +
```

```
    calculateSalary());
```

```
}
```

```
}
```

```
public class EmployeePayrollManagementSystem {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        try {
```

```
            int managerId = sc.nextInt();
```

```
            sc.nextLine();
```

```
            String managerName = sc.nextLine();
```

```
            double managerSalary = sc.nextDouble();
```

```
            int managerDays = sc.nextInt();
```

```
            double incentive = sc.nextDouble();
```

```
            Manager m1 = new Manager(
```

```
        managerId,  
        managerName,  
        managerSalary,  
        managerDays,  
        incentive  
    );  
  
    m1.displayManager();  
  
    int clerkId = sc.nextInt();  
    sc.nextLine();  
  
    String clerkName = sc.nextLine();  
  
    double clerkSalary = sc.nextDouble();  
  
    int clerkDays = sc.nextInt();  
  
    Clerk c1 = new Clerk(  
        clerkId,  
        clerkName,  
        clerkSalary,  
        clerkDays  
    );  
  
    c1.displayClerk();  
  
    System.out.println("\nTotal Employees: " +  
        Employee.employeeCount);
```

```

    }

    catch(IllegalArgumentException e) {

        System.out.println("Exception: " + e.getMessage());
    }

    catch(Exception e) {

        System.out.println("Unexpected Error: " + e.getMessage());
    }

    finally {

        System.out.println("\nProgram Executed Successfully");
    }

    sc.close();
}
}

```

5.Library Management System

- Use classes `Book`, `Member`, `Librarian`, and demonstrate method overriding.
- Package: `com.library.system`; Interface: `Borrowable`.
- Logic for issuing, returning books, and fine calculation.
 - Custom exceptions like `BookUnavailableException`, `FineCalculationException`.

(Use constructors, static, this, and final keywords wherever applicable)

Code :

```
package com.library.system;
```

```
import java.util.Scanner;
```

```
interface Borrowable {  
    void issueBook();  
    void returnBook(int lateDays);  
}
```

```
class Book {
```

```
    protected int bookId;  
    protected String bookName;  
    protected String author;  
    protected boolean isAvailable;
```

```
    static int totalBooksIssued = 0;
```

```
    final String libraryName = "Central Library";
```

```
    public Book(int bookId,  
                String bookName,  
                String author) {
```

```
        this.bookId = bookId;  
        this.bookName = bookName;  
        this.author = author;  
        this.isAvailable = true;  
    }
```

```
    public void displayBook() {
```

```
        System.out.println("Library Name: " + libraryName);
        System.out.println("Book ID: " + bookId);
        System.out.println("Book Name: " + bookName);
        System.out.println("Author: " + author);
        System.out.println("Available: " + isAvailable);
    }
}
```

```
class Member extends Book implements Borrowable {
```

```
    protected String memberName;
```

```
    protected int memberId;
```

```
    protected double fine;
```

```
    public Member(int bookId,
                  String bookName,
                  String author,
                  String memberName,
                  int memberId) {
```

```
        super(bookId, bookName, author);
```

```
        if(memberName == null ||
           memberName.trim().isEmpty()) {
```

```
            throw new IllegalArgumentException(
                "Member name cannot be empty"
            );
```

```
}
```

```
    this.memberName = memberName;
```

```
    this.memberId = memberId;
```

```
}
```

```
@Override
```

```
public void issueBook() {
```

```
    if(!isAvailable) {
```

```
        throw new BookUnavailableException(
```

```
            "Book is currently unavailable"
```

```
        );
```

```
    }
```

```
    isAvailable = false;
```

```
    totalBooksIssued++;
```

```
    System.out.println("Book Issued Successfully");
```

```
}
```

```
@Override
```

```
public void returnBook(int lateDays) {
```

```
    isAvailable = true;
```

```
    calculateFine(lateDays);
```

```
totalBooksIssued--;

    System.out.println("Book Returned Successfully");
}

public void calculateFine(int lateDays) {

    if(lateDays < 0) {

        throw new FineCalculationException(
            "Late days cannot be negative"
        );
    }

    fine = lateDays * 10;
}

public void displayMember() {

    System.out.println("\n===== MEMBER DETAILS =====");

    displayBook();

    System.out.println("Member Name: " + memberName);
    System.out.println("Member ID: " + memberId);
    System.out.println("Fine: " + fine);
}
}
```

```

class Librarian extends Member {

    public Librarian(int bookId,
                     String bookName,
                     String author,
                     String memberName,
                     int memberId) {

        super(bookId,
              bookName,
              author,
              memberName,
              memberId);
    }

    @Override
    public void displayMember() {

        System.out.println("\n===== LIBRARIAN VIEW =====");

        super.displayMember();

        System.out.println("Managed By Librarian");
    }
}

class BookUnavailableException
    extends RuntimeException {

```

```
public BookUnavailableException(String message) {  
    super(message);  
}  
}
```

```
class FineCalculationException  
    extends RuntimeException {  
  
    public FineCalculationException(String message) {  
        super(message);  
    }  
}
```

```
public class LibraryManagementSystem {  
  
    public static void main(String[] args) {  
  
        Scanner sc = new Scanner(System.in);  
  
        try {  
  
            int bookId = sc.nextInt();  
            sc.nextLine();  
  
            String bookName = sc.nextLine();  
  
            String author = sc.nextLine();  
  
            String memberName = sc.nextLine();  

```

```
int memberId = sc.nextInt();

int lateDays = sc.nextInt();

Librarian l1 = new Librarian(
    bookId,
    bookName,
    author,
    memberName,
    memberId
);

l1.issueBook();

l1.returnBook(lateDays);

l1.displayMember();

System.out.println("\nTotal Books Issued: " +
    Book.totalBooksIssued);
}

catch(BookUnavailableException e) {

    System.out.println("Book Exception: " +
        e.getMessage());
}
```

```

        catch(FineCalculationException e) {

            System.out.println("Fine Exception: " +
                e.getMessage());
        }

        catch(Exception e) {

            System.out.println("Unexpected Error: " +
                e.getMessage());
        }

        finally {

            System.out.println("\nProgram Executed Successfully");
        }

        sc.close();
    }
}

```

6. Banking System with Transaction History

- Use classes like `Account`, `SavingsAccount`, `Transaction`.
- Package: `com.bank.accounts`; Interface: `Transactable`.
- Implement logic for deposit, withdrawal, balance check, and transaction history using collections.

- Handle **insufficient balance**, **invalid transactions**, and **duplicate transaction entries**.

(Use constructors, static, this, and final keywords wherever applicable)

Code :

```
package com.bank.accounts;
```

```
import java.util.ArrayList;
```

```
import java.util.HashSet;
```

```
import java.util.Scanner;
```

```
interface Transactable {
```

```
    void deposit(double amount);
```

```
    void withdraw(double amount);
```

```
}
```

```
class Transaction {
```

```
    private int transactionId;
```

```
    private String transactionType;
```

```
    private double amount;
```

```
    public Transaction(int transactionId,
```

```
                        String transactionType,
```

```
                        double amount) {
```

```
        if(amount <= 0) {
```

```
            throw new IllegalArgumentException(
```

```
                "Transaction amount must be positive"
```

```
            );
```

```
        }
```

```
        this.transactionId = transactionId;
```

```
        this.transactionType = transactionType;
```

```
    this.amount = amount;
}
```

```
public int getTransactionId() {
    return transactionId;
}
```

```
@Override
```

```
public String toString() {
```

```
    return "Transaction ID: " + transactionId +
        ", Type: " + transactionType +
        ", Amount: " + amount;
```

```
    }
}
```

```
class Account {
```

```
    protected int accountNumber;
    protected String accountHolder;
    protected double balance;
```

```
    static int totalAccounts = 0;
```

```
    final String bankName = "State Bank";
```

```
    public Account(int accountNumber,
        String accountHolder,
        double balance) {
```

```
if(accountHolder == null ||
    accountHolder.trim().isEmpty()) {

    throw new IllegalArgumentException(
        "Account holder name cannot be empty"
    );
}

if(balance < 0) {

    throw new IllegalArgumentException(
        "Initial balance cannot be negative"
    );
}

this.accountNumber = accountNumber;
this.accountHolder = accountHolder;
this.balance = balance;

totalAccounts++;
}

public void displayAccount() {

    System.out.println("Bank Name: " + bankName);
    System.out.println("Account Number: " + accountNumber);
    System.out.println("Account Holder: " + accountHolder);
    System.out.println("Balance: " + balance);
}
```

```
}  
}
```

```
class SavingsAccount extends Account  
    implements Transactable {  
  
    private ArrayList<Transaction> history;  
  
    private HashSet<Integer> transactionIds;  
  
    public SavingsAccount(int accountNumber,  
        String accountHolder,  
        double balance) {  
  
        super(accountNumber,  
            accountHolder,  
            balance);  
  
        history = new ArrayList<>();  
  
        transactionIds = new HashSet<>();  
    }
```

```
@Override  
public void deposit(double amount) {  
  
    if(amount <= 0) {  
  
        throw new IllegalArgumentException(  

```

```
        "Deposit amount must be positive"
    );
}

balance += amount;

System.out.println("Amount Deposited Successfully");
}

@Override
public void withdraw(double amount) {

    if(amount <= 0) {

        throw new IllegalArgumentException(
            "Withdrawal amount must be positive"
        );
    }

    if(amount > balance) {

        throw new IllegalArgumentException(
            "Insufficient Balance"
        );
    }

    balance -= amount;

    System.out.println("Amount Withdrawn Successfully");
```

```
}
```

```
public void addTransaction(Transaction t) {
```

```
    if(transactionIds.contains(  
        t.getTransactionId())) {
```

```
        throw new IllegalArgumentException(  
            "Duplicate Transaction Entry"  
        );
```

```
    }
```

```
    transactionIds.add(t.getTransactionId());
```

```
    history.add(t);
```

```
}
```

```
public void showTransactionHistory() {
```

```
    System.out.println("\n===== TRANSACTION HISTORY =====");
```

```
    for(Transaction t : history) {
```

```
        System.out.println(t);
```

```
    }
```

```
}
```

```
public void checkBalance() {
```

```
    System.out.println("Current Balance: " +
```

```
        balance);  
    }  
}
```

```
public class BankingSystem {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        try {
```

```
            int accNo = sc.nextInt();
```

```
            sc.nextLine();
```

```
            String holderName = sc.nextLine();
```

```
            double balance = sc.nextDouble();
```

```
            SavingsAccount s1 =
```

```
                new SavingsAccount(
```

```
                    accNo,
```

```
                    holderName,
```

```
                    balance
```

```
                );
```

```
            int depositId = sc.nextInt();
```

```
            double depositAmount = sc.nextDouble();
```

```
s1.deposit(depositAmount);
```

```
Transaction t1 =
```

```
    new Transaction(  
        depositId,  
        "Deposit",  
        depositAmount  
    );
```

```
s1.addTransaction(t1);
```

```
int withdrawId = sc.nextInt();
```

```
double withdrawAmount =  
    sc.nextDouble();
```

```
s1.withdraw(withdrawAmount);
```

```
Transaction t2 =
```

```
    new Transaction(  
        withdrawId,  
        "Withdraw",  
        withdrawAmount  
    );
```

```
s1.addTransaction(t2);
```

```
System.out.println(
```

```

        "\n===== ACCOUNT DETAILS ====="
    );

    s1.displayAccount();

    s1.checkBalance();

    s1.showTransactionHistory();

    System.out.println(
        "\nTotal Accounts: " +
        Account.totalAccounts
    );
}

catch(IllegalArgumentException e) {

    System.out.println(
        "Exception: " +
        e.getMessage()
    );
}

catch(Exception e) {

    System.out.println(
        "Unexpected Error: " +
        e.getMessage()
    );
}

```

```

    }

    finally {

        System.out.println(

            "\nProgram Executed Successfully"

        );

    }

    sc.close();

}
}

```

7.Vehicle Rental System

- Define base class `Vehicle`, with `Car`, `Bike` subclasses.
- Interface: `Rentable` with method `calculateRent()`.
- Package: `com.rental.vehicles`.
- Include logic to calculate rent based on duration and type.
 - Manage exceptions like **invalid rental duration**, **vehicle unavailability**, or **invalid input**.

(Use constructors, static, this, and final keywords wherever applicable)

Code :

```

package com.rental.vehicles;

import java.util.Scanner;

interface Rentable {

    double calculateRent(int days);

}

class Vehicle {

```

```
protected int vehicleId;

protected String vehicleName;

protected boolean isAvailable;


static int totalVehiclesRented = 0;


final String companyName = "City Vehicle Rentals";


public Vehicle(int vehicleId,
               String vehicleName,
               boolean isAvailable) {

    if(vehicleName == null ||
       vehicleName.trim().isEmpty()) {

        throw new IllegalArgumentException(
            "Vehicle name cannot be empty"
        );
    }

    this.vehicleId = vehicleId;
    this.vehicleName = vehicleName;
    this.isAvailable = isAvailable;
}


public void displayVehicle() {

    System.out.println("Company Name: " + companyName);
```

```
        System.out.println("Vehicle ID: " + vehicleId);
        System.out.println("Vehicle Name: " + vehicleName);
        System.out.println("Available: " + isAvailable);
    }
}
```

```
class Car extends Vehicle implements Rentable {
```

```
    private double rentPerDay;
```

```
    public Car(int vehicleId,
               String vehicleName,
               boolean isAvailable,
               double rentPerDay) {
```

```
        super(vehicleId,
               vehicleName,
               isAvailable);
```

```
        if(rentPerDay <= 0) {
```

```
            throw new IllegalArgumentException(
                "Rent per day must be positive"
            );
        }
```

```
        this.rentPerDay = rentPerDay;
    }
```

```
@Override
public double calculateRent(int days) {

    if(days <= 0) {

        throw new IllegalArgumentException(
            "Invalid rental duration"
        );
    }

    if(!isAvailable) {

        throw new IllegalArgumentException(
            "Vehicle unavailable"
        );
    }

    totalVehiclesRented++;

    return rentPerDay * days;
}

public void displayCar(int days) {

    System.out.println("\n===== CAR DETAILS =====");

    displayVehicle();

    System.out.println("Rent Per Day: " + rentPerDay);
```

```
        System.out.println("Total Rent: " +  
            calculateRent(days));  
    }  
}
```

```
class Bike extends Vehicle implements Rentable {
```

```
    private double rentPerDay;
```

```
    public Bike(int vehicleId,  
        String vehicleName,  
        boolean isAvailable,  
        double rentPerDay) {
```

```
        super(vehicleId,  
            vehicleName,  
            isAvailable);
```

```
        if(rentPerDay <= 0) {
```

```
            throw new IllegalArgumentException(  
                "Rent per day must be positive"  
            );  
        }
```

```
        this.rentPerDay = rentPerDay;  
    }
```

```
@Override
public double calculateRent(int days) {

    if(days <= 0) {

        throw new IllegalArgumentException(
            "Invalid rental duration"
        );
    }

    if(!isAvailable) {

        throw new IllegalArgumentException(
            "Vehicle unavailable"
        );
    }

    totalVehiclesRented++;

    return rentPerDay * days;
}

public void displayBike(int days) {

    System.out.println("\n===== BIKE DETAILS =====");

    displayVehicle();

    System.out.println("Rent Per Day: " + rentPerDay);
```

```
        System.out.println("Total Rent: " +  
            calculateRent(days));  
    }  
}
```

```
public class VehicleRentalSystem {  
  
    public static void main(String[] args) {  
  
        Scanner sc = new Scanner(System.in);  
  
        try {  
  
            int carId = sc.nextInt();  
            sc.nextLine();  
  
            String carName = sc.nextLine();  
  
            boolean carAvailable = sc.nextBoolean();  
  
            double carRent = sc.nextDouble();  
  
            int carDays = sc.nextInt();  
  
            Car c1 = new Car(  
                carId,  
                carName,  
                carAvailable,
```

```
        carRent
    );

    c1.displayCar(carDays);

    sc.nextLine();

    int bikeId = sc.nextInt();
    sc.nextLine();

    String bikeName = sc.nextLine();

    boolean bikeAvailable = sc.nextBoolean();

    double bikeRent = sc.nextDouble();

    int bikeDays = sc.nextInt();

    Bike b1 = new Bike(
        bikeId,
        bikeName,
        bikeAvailable,
        bikeRent
    );

    b1.displayBike(bikeDays);

    System.out.println(
        "\nTotal Vehicles Rented: " +
```

```
        Vehicle.totalVehiclesRented

    );
}

catch(IllegalArgumentException e) {

    System.out.println(

        "Exception: " +

        e.getMessage()

    );
}

catch(Exception e) {

    System.out.println(

        "Unexpected Error: " +

        e.getMessage()

    );
}

finally {

    System.out.println(

        "\nProgram Executed Successfully"

    );
}

sc.close();
}
```

```
}
```

8.Inventory Management System

- Define `Inventory`, `Item`, `Warehouse` classes.
- Interface: `Storable`, Package: `com.inventory.stock`.
- Logic to track stock levels and generate reorder alerts.
 - Catch **stock underflow**, **duplicate item codes**, or **null item references**.

(Use constructors, static, this, and final keywords wherever applicable)

Code :

```
package com.inventory.stock;
```

```
import java.util.ArrayList;
```

```
import java.util.HashSet;
```

```
import java.util.Scanner;
```

```
interface Storable {
```

```
    void addStock(int quantity);
```

```
    void removeStock(int quantity);
```

```
}
```

```
class Item {
```

```
    protected int itemCode;
```

```
    protected String itemName;
```

```
    protected int stock;
```

```
    static int totalItems = 0;
```

```
    final String inventoryName = "Central Inventory";
```

```
public Item(int itemCode,
            String itemName,
            int stock) {

    if(itemName == null ||
        itemName.trim().isEmpty()) {

        throw new IllegalArgumentException(
            "Item name cannot be null"
        );
    }

    if(stock < 0) {

        throw new IllegalArgumentException(
            "Stock cannot be negative"
        );
    }

    this.itemCode = itemCode;
    this.itemName = itemName;
    this.stock = stock;

    totalItems++;
}

public void displayItem() {

    System.out.println("Inventory Name: " +
```

```

        inventoryName);

        System.out.println("Item Code: " +
            itemCode);

        System.out.println("Item Name: " +
            itemName);

        System.out.println("Stock: " + stock);
    }
}

class Inventory extends Item
    implements Storable {

    private static HashSet<Integer> itemCodes =
        new HashSet<>();

    public Inventory(int itemCode,
        String itemName,
        int stock) {

        super(itemCode,
            itemName,
            stock);

        if(itemCodes.contains(itemCode)) {

            throw new IllegalArgumentException(

```

```

        "Duplicate Item Code"

    );
}

    itemCodes.add(itemCode);
}

@Override
public void addStock(int quantity) {

    if(quantity <= 0) {

        throw new IllegalArgumentException(
            "Invalid stock quantity"
        );
    }

    stock += quantity;

    System.out.println(
        "Stock Added Successfully"
    );
}

@Override
public void removeStock(int quantity) {

    if(quantity <= 0) {

```

```
        throw new IllegalArgumentException(
            "Invalid stock quantity"
        );
    }

    if(quantity > stock) {

        throw new IllegalArgumentException(
            "Stock Underflow"
        );
    }

    stock -= quantity;

    System.out.println(
        "Stock Removed Successfully"
    );
}

public void reorderAlert() {

    if(stock < 10) {

        System.out.println(
            "Reorder Alert: Low Stock"
        );
    }
}
}
```

```
class Warehouse {

    private ArrayList<Inventory> inventoryList;

    public Warehouse() {

        inventoryList = new ArrayList<>();
    }

    public void addItem(Inventory item) {

        if(item == null) {

            throw new IllegalArgumentException(

                "Null Item Reference"

            );

        }

        inventoryList.add(item);

    }

    public void displayWarehouse() {

        System.out.println(

            "\n===== WAREHOUSE DETAILS ====="

        );

        for(Inventory item : inventoryList) {
```

```

        item.displayItem();

        item.reorderAlert();

        System.out.println(
            "-----"
        );
    }
}

public class InventoryManagementSystem {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        try {

            Warehouse warehouse =
                new Warehouse();

            int itemCode = sc.nextInt();
            sc.nextLine();

            String itemName =
                sc.nextLine();

```

```
int stock = sc.nextInt();

Inventory i1 =
    new Inventory(
        itemCode,
        itemName,
        stock
    );

warehouse.addItem(i1);

int addQuantity =
    sc.nextInt();

i1.addStock(addQuantity);

int removeQuantity =
    sc.nextInt();

i1.removeStock(removeQuantity);

warehouse.displayWarehouse();

System.out.println(
    "\nTotal Items: " +
    Item.totalItems
);
}
```

```

        catch(IllegalArgumentException e) {

            System.out.println(
                "Exception: " +
                e.getMessage()
            );
        }

        catch(Exception e) {

            System.out.println(
                "Unexpected Error: " +
                e.getMessage()
            );
        }

        finally {

            System.out.println(
                "\nProgram Executed Successfully"
            );
        }

        sc.close();
    }
}

```

9. Hospital Appointment Booking System

► Create classes for Doctor, Patient, Appointment.

- Interface: `Schedulable`, Package: `com.hospital.appointments`.
- Logic for time-slot checking and appointment booking.

Handle exceptions like **overlapping appointments**, **invalid time slots**, or **null patient info**.

(Use constructors, static, this, and final keywords wherever applicable)

Code :

```
package com.hospital.appointments;
```

```
import java.util.ArrayList;
```

```
import java.util.Scanner;
```

```
interface Schedulable {  
    void bookAppointment(String slot);  
}
```

```
class Doctor {  
  
    protected int doctorId;  
    protected String doctorName;  
    protected String specialization;  
  
    static int totalAppointments = 0;  
  
    final String hospitalName = "City Hospital";  
  
    public Doctor(int doctorId,  
                  String doctorName,  
                  String specialization) {  
  
        if(doctorName == null ||
```

```
        doctorName.trim().isEmpty()) {

            throw new IllegalArgumentException(

                "Doctor name cannot be null"

            );

        }

        this.doctorId = doctorId;

        this.doctorName = doctorName;

        this.specialization = specialization;

    }

    public void displayDoctor() {

        System.out.println("Hospital Name: " +

            hospitalName);

        System.out.println("Doctor ID: " +

            doctorId);

        System.out.println("Doctor Name: " +

            doctorName);

        System.out.println("Specialization: " +

            specialization);

    }

}

class Patient extends Doctor {
```

```
protected int patientId;
```

```
protected String patientName;
```

```
public Patient(int doctorId,  
               String doctorName,  
               String specialization,  
               int patientId,  
               String patientName) {
```

```
    super(doctorId,  
          doctorName,  
          specialization);
```

```
    if(patientName == null ||  
       patientName.trim().isEmpty()) {
```

```
        throw new IllegalArgumentException(  
            "Patient information cannot be null"  
        );  
    }
```

```
    this.patientId = patientId;  
    this.patientName = patientName;  
}
```

```
public void displayPatient() {
```

```
    System.out.println("Patient ID: " +
```

```

        patientId);

        System.out.println("Patient Name: " +
            patientName);
    }
}

class Appointment extends Patient
    implements Schedulable {

    private String appointmentSlot;

    private static ArrayList<String> bookedSlots =
        new ArrayList<>();

    public Appointment(int doctorId,
        String doctorName,
        String specialization,
        int patientId,
        String patientName) {

        super(doctorId,
            doctorName,
            specialization,
            patientId,
            patientName);
    }
}

```

@Override

```
public void bookAppointment(String slot) {

    if(slot == null ||
        slot.trim().isEmpty()) {

        throw new IllegalArgumentException(
            "Invalid time slot"
        );
    }

    if(bookedSlots.contains(slot)) {

        throw new IllegalArgumentException(
            "Overlapping appointment detected"
        );
    }

    this.appointmentSlot = slot;

    bookedSlots.add(slot);

    totalAppointments++;

    System.out.println(
        "Appointment Booked Successfully"
    );
}

public void displayAppointment() {
```

```
System.out.println(
    "\n===== APPOINTMENT DETAILS ====="
);

displayDoctor();

displayPatient();

System.out.println(
    "Appointment Slot: " +
    appointmentSlot
);

System.out.println(
    "Total Appointments: " +
    totalAppointments
);
}
}

public class HospitalAppointmentBookingSystem {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        try {
```

```
int doctorId = sc.nextInt();  
sc.nextLine();
```

```
String doctorName =  
    sc.nextLine();
```

```
String specialization =  
    sc.nextLine();
```

```
int patientId =  
    sc.nextInt();  
sc.nextLine();
```

```
String patientName =  
    sc.nextLine();
```

```
String slot =  
    sc.nextLine();
```

```
Appointment a1 =  
    new Appointment(  
        doctorId,  
        doctorName,  
        specialization,  
        patientId,  
        patientName  
    );
```

```
a1.bookAppointment(slot);
```

```
        a1.displayAppointment();
    }

    catch(IllegalArgumentException e) {

        System.out.println(
            "Exception: " +
            e.getMessage()
        );
    }

    catch(Exception e) {

        System.out.println(
            "Unexpected Error: " +
            e.getMessage()
        );
    }

    finally {

        System.out.println(
            "\nProgram Executed Successfully"
        );
    }

    sc.close();
}
```

```
}
```

10.Smart Home Device Control System

- Create `Device` base class, and subclasses like `Light`, `Fan`.
- Use interfaces `Switchable`, `Configurable`.
- Package: `com.smarthome.devices`.
- Logic to turn devices on/off and change settings.

Manage exceptions like **device offline errors**, **invalid configuration values**.

(Use constructors, static, this, and final keywords wherever applicable)

Code :

```
package com.smarthome.devices;
```

```
import java.util.Scanner;
```

```
interface Switchable {  
  
    void turnOn();  
  
    void turnOff();  
  
}
```

```
interface Configurable {  
  
    void configure(int value);  
  
}
```

```
class Device {  
  
    protected int deviceId;
```

```
protected String deviceName;
```

```
protected boolean isOnline;
```

```
protected boolean isOn;
```

```
static int totalDevices = 0;
```

```
final String companyName = "Smart Home Solutions";
```

```
public Device(int deviceId,  
              String deviceName,  
              boolean isOnline) {
```

```
    if(deviceName == null ||  
        deviceName.trim().isEmpty()) {
```

```
        throw new IllegalArgumentException(  
            "Device name cannot be null"  
        );
```

```
    }
```

```
    this.deviceId = deviceId;
```

```
    this.deviceName = deviceName;
```

```
    this.isOnline = isOnline;
```

```
    this.isOn = false;
```

```
        totalDevices++;
    }

    public void displayDevice() {

        System.out.println("Company Name: " +
            companyName);

        System.out.println("Device ID: " +
            deviceId);

        System.out.println("Device Name: " +
            deviceName);

        System.out.println("Online Status: " +
            isOnline);

        System.out.println("Power Status: " +
            isOn);
    }
}

class Light extends Device
```

```
implements Switchable, Configurable {

private int brightness;

public Light(int deviceId,
             String deviceName,
             boolean isOnline) {

    super(deviceId,
           deviceName,
           isOnline);

    brightness = 0;
}

@Override
public void turnOn() {

    if(!isOnline) {

        throw new IllegalArgumentException(
            "Device Offline Error"
        );
    }
}
```

```
isOn = true;
```

```
System.out.println(
```

```
    "Light Turned ON"
```

```
);
```

```
}
```

```
@Override
```

```
public void turnOff() {
```

```
    if(!isOnline) {
```

```
        throw new IllegalArgumentException(
```

```
            "Device Offline Error"
```

```
        );
```

```
    }
```

```
isOn = false;
```

```
System.out.println(
```

```
    "Light Turned OFF"
```

```
);
```

```
}
```

```
@Override
```

```
public void configure(int value) {
```

```
    if(value < 0 || value > 100) {
```

```
        throw new IllegalArgumentException(
```

```
            "Invalid Brightness Value"
```

```
        );
```

```
    }
```

```
    brightness = value;
```

```
    System.out.println(
```

```
        "Brightness Set To: " +
```

```
        brightness
```

```
    );
```

```
}
```

```
public void displayLight() {
```

```
    System.out.println(
```

```
        "\n===== LIGHT DETAILS ====="
```

```
    );
```

```
displayDevice();
```

```
System.out.println(
```

```
    "Brightness: " +
```

```
    brightness
```

```
);
```

```
}
```

```
}
```

```
class Fan extends Device
```

```
    implements Switchable, Configurable {
```

```
    private int speed;
```

```
    public Fan(int deviceId,
```

```
        String deviceName,
```

```
        boolean isOnline) {
```

```
        super(deviceId,
```

```
            deviceName,
```

```
            isOnline);
```

```
        speed = 0;
```

```
}
```

```
@Override
```

```
public void turnOn() {
```

```
    if(!isOnline) {
```

```
        throw new IllegalArgumentException(
```

```
            "Device Offline Error"
```

```
        );
```

```
    }
```

```
    isOn = true;
```

```
    System.out.println(
```

```
        "Fan Turned ON"
```

```
    );
```

```
}
```

```
@Override
```

```
public void turnOff() {
```

```
    if(!isOnline) {
```

```
        throw new IllegalArgumentException(  
            "Device Offline Error"  
        );  
    }
```

```
    isOn = false;
```

```
    System.out.println(  
        "Fan Turned OFF"  
    );  
}
```

```
@Override  
public void configure(int value) {
```

```
    if(value < 1 || value > 5) {
```

```
        throw new IllegalArgumentException(  
            "Invalid Fan Speed"  
        );  
    }
```

```
    speed = value;
```

```
System.out.println(  
    "Fan Speed Set To: " +  
    speed  
);  
}
```

```
public void displayFan() {
```

```
    System.out.println(  
        "\n===== FAN DETAILS ====="  
    );
```

```
    displayDevice();
```

```
    System.out.println(  
        "Fan Speed: " +  
        speed  
    );  
}  
}
```

```
public class SmartHomeDeviceControlSystem {
```

```
    public static void main(String[] args) {
```

```
Scanner sc = new Scanner(System.in);
```

```
try {
```

```
    int lightId = sc.nextInt();
```

```
    sc.nextLine();
```

```
    String lightName =
```

```
        sc.nextLine();
```

```
    boolean lightOnline =
```

```
        sc.nextBoolean();
```

```
    int brightness =
```

```
        sc.nextInt();
```

```
    Light l1 =
```

```
        new Light(
```

```
            lightId,
```

```
            lightName,
```

```
            lightOnline
```

```
        );
```

```
l1.turnOn();
```

```
l1.configure(brightness);
```

```
l1.displayLight();
```

```
int fanId =
```

```
    sc.nextInt();
```

```
sc.nextLine();
```

```
String fanName =
```

```
    sc.nextLine();
```

```
boolean fanOnline =
```

```
    sc.nextBoolean();
```

```
int speed =
```

```
    sc.nextInt();
```

```
Fan f1 =
```

```
    new Fan(
```

```
        fanId,
```

```
        fanName,
```

```
        fanOnline
```

```
);
```

```
f1.turnOn();
```

```
f1.configure(speed);
```

```
f1.displayFan();
```

```
System.out.println(
```

```
    "\nTotal Devices: " +
```

```
    Device.totalDevices
```

```
);
```

```
}
```

```
catch(IllegalArgumentException e) {
```

```
    System.out.println(
```

```
        "Exception: " +
```

```
        e.getMessage()
```

```
);
```

```
}
```

```
catch(Exception e) {
```

```

        System.out.println(

            "Unexpected Error: " +

            e.getMessage()

        );
    }

    finally {

        System.out.println(

            "\nProgram Executed Successfully"

        );
    }

    sc.close();
}
}

```

11.Airline Ticketing System

- Classes: `Flight`, `Passenger`, `Ticket`. Use inheritance and polymorphism.
- Package: `com.airline.booking`, Interface: `CheckIn`.
- Logic for seat availability, booking, and fare calculation.
 - Handle exceptions like **double bookings**, **invalid passenger details**, **overbooked flights**.

(Use constructors, static, this, and final keywords wherever applicable)

Code :

```
package com.airline.booking;
```

```
import java.util.ArrayList;

import java.util.Scanner;

interface CheckIn {

    void checkIn();

}

class Flight {

    protected int flightNumber;

    protected String flightName;

    protected int totalSeats;

    protected int availableSeats;

    static int totalBookings = 0;

    final String airlineName = "Sky Airlines";

    public Flight(int flightNumber,

                  String flightName,

                  int totalSeats) {

        if(flightName == null ||

           flightName.trim().isEmpty()) {

            throw new IllegalArgumentException(

                "Flight name cannot be empty"

            );

        }

    }

}
```

```
if(totalSeats <= 0) {

    throw new IllegalArgumentException(
        "Invalid seat count"
    );
}

this.flightNumber = flightNumber;
this.flightName = flightName;
this.totalSeats = totalSeats;
this.availableSeats = totalSeats;
}

public void displayFlight() {

    System.out.println("Airline Name: " +
        airlineName);

    System.out.println("Flight Number: " +
        flightNumber);

    System.out.println("Flight Name: " +
        flightName);

    System.out.println("Available Seats: " +
        availableSeats);
}
}
```

```
class Passenger extends Flight {

    protected int passengerId;
    protected String passengerName;

    public Passenger(int flightNumber,
                     String flightName,
                     int totalSeats,
                     int passengerId,
                     String passengerName) {

        super(flightNumber,
              flightName,
              totalSeats);

        if(passengerName == null ||
           passengerName.trim().isEmpty()) {

            throw new IllegalArgumentException(
                "Invalid passenger details"
            );
        }

        this.passengerId = passengerId;
        this.passengerName = passengerName;
    }

    public void displayPassenger() {
```

```

        System.out.println("Passenger ID: " +
            passengerId);

        System.out.println("Passenger Name: " +
            passengerName);
    }
}

class Ticket extends Passenger
    implements CheckIn {

    private int seatsBooked;
    private double fare;

    private static ArrayList<Integer> bookedPassengers =
        new ArrayList<>();

    public Ticket(int flightNumber,
        String flightName,
        int totalSeats,
        int passengerId,
        String passengerName,
        int seatsBooked) {

        super(flightNumber,
            flightName,
            totalSeats,
            passengerId,

```

```
        passengerName);

    if(seatsBooked <= 0) {

        throw new IllegalArgumentException(
            "Invalid seat booking"
        );
    }

    this.seatsBooked = seatsBooked;

    calculateFare();
}

public void calculateFare() {

    fare = seatsBooked * 5000;
}

public void bookTicket() {

    if(bookedPassengers.contains(
        passengerId)) {

        throw new IllegalArgumentException(
            "Double booking not allowed"
        );
    }
}
```

```
if(seatsBooked > availableSeats) {  
  
    throw new IllegalArgumentException(  
        "Flight Overbooked"  
    );  
}  
  
availableSeats -= seatsBooked;  
  
bookedPassengers.add(passengerId);  
  
totalBookings++;  
  
System.out.println(  
    "Ticket Booked Successfully"  
);  
}
```

```
@Override  
public void checkIn() {  
  
    System.out.println(  
        "Passenger Checked In Successfully"  
    );  
}
```

```
public void displayTicket() {  
  
    System.out.println(  

```

```

        "\n===== TICKET DETAILS ====="
    );

    displayFlight();

    displayPassenger();

    System.out.println(
        "Seats Booked: " +
        seatsBooked
    );

    System.out.println(
        "Fare: " +
        fare
    );

    System.out.println(
        "Total Bookings: " +
        totalBookings
    );
}

}

public class AirlineTicketingSystem {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
    }
}

```

```
try {

    int flightNumber =
        sc.nextInt();
    sc.nextLine();

    String flightName =
        sc.nextLine();

    int totalSeats =
        sc.nextInt();

    int passengerId =
        sc.nextInt();
    sc.nextLine();

    String passengerName =
        sc.nextLine();

    int seatsBooked =
        sc.nextInt();

    Ticket t1 =
        new Ticket(
            flightNumber,
            flightName,
            totalSeats,
            passengerId,
```

```
        passengerName,  
        seatsBooked  
    );  
  
    t1.bookTicket();  
  
    t1.checkIn();  
  
    t1.displayTicket();  
}  
  
catch(IllegalArgumentException e) {  
  
    System.out.println(  
        "Exception: " +  
        e.getMessage()  
    );  
}  
  
catch(Exception e) {  
  
    System.out.println(  
        "Unexpected Error: " +  
        e.getMessage()  
    );  
}  
  
finally {
```

```

        System.out.println(
            "\nProgram Executed Successfully"
        );
    }

    sc.close();
}
}

```

12. Online Quiz Management System

- Define `Question`, `User`, and `Quiz` classes.
- Interface: `Scorable`, Package: `com.quiz.app`.
- Logic to present quiz, accept answers, and calculate score.
 - Manage Exceptions like **invalid input format**, **unanswered questions**, **score overflow**.

(Use constructors, static, this, and final keywords wherever applicable)

Code :

```
package com.quiz.app;
```

```
import java.util.Scanner;
```

```
interface Scorable {
    int calculateScore();
}

```

```
class Question {

    protected int questionId;
    protected String questionText;
    protected String correctAnswer;
}

```

```
static int totalQuestions = 0;
```

```
final String appName = "Online Quiz App";
```

```
public Question(int questionId,  
                String questionText,  
                String correctAnswer) {
```

```
    if(questionText == null ||  
        questionText.trim().isEmpty()) {
```

```
        throw new IllegalArgumentException(  
            "Question cannot be empty"  
        );  
    }
```

```
    if(correctAnswer == null ||  
        correctAnswer.trim().isEmpty()) {
```

```
        throw new IllegalArgumentException(  
            "Correct answer cannot be empty"  
        );  
    }
```

```
    this.questionId = questionId;
```

```
    this.questionText = questionText;
```

```
    this.correctAnswer = correctAnswer;
```

```
    totalQuestions++;
```

```
}
```

```
public void displayQuestion() {
```

```
    System.out.println("App Name: " +  
        appName);
```

```
    System.out.println("Question ID: " +  
        questionId);
```

```
    System.out.println("Question: " +  
        questionText);
```

```
}
```

```
}
```

```
class User extends Question {
```

```
    protected int userId;  
    protected String userName;  
    protected String userAnswer;
```

```
    public User(int questionId,  
        String questionText,  
        String correctAnswer,  
        int userId,  
        String userName,  
        String userAnswer) {
```

```
        super(questionId,
```

```
        questionText,
        correctAnswer);

    if(userName == null ||
        userName.trim().isEmpty()) {

        throw new IllegalArgumentException(
            "Invalid user name"
        );
    }

    if(userAnswer == null ||
        userAnswer.trim().isEmpty()) {

        throw new IllegalArgumentException(
            "Unanswered Question"
        );
    }

    this.userId = userId;
    this.userName = userName;
    this.userAnswer = userAnswer;
}

public void displayUser() {

    System.out.println("User ID: " +
        userId);
}
```

```
        System.out.println("User Name: " +
            userName);

        System.out.println("User Answer: " +
            userAnswer);
    }
}
```

```
class Quiz extends User
    implements Scorable {

    private int score;

    public Quiz(int questionId,
        String questionText,
        String correctAnswer,
        int userId,
        String userName,
        String userAnswer) {

        super(questionId,
            questionText,
            correctAnswer,
            userId,
            userName,
            userAnswer);
    }
```

@Override

```
public int calculateScore() {  
  
    if(score > Integer.MAX_VALUE - 10) {  
  
        throw new ArithmeticException(  
            "Score Overflow"  
        );  
    }  
  
    if(userAnswer.equalsIgnoreCase(  
        correctAnswer)) {  
  
        score += 10;  
    }  
  
    return score;  
}
```

```
public void displayQuiz() {  
  
    System.out.println(  
        "\n===== QUIZ DETAILS ====="  
    );  
  
    displayQuestion();  
  
    displayUser();  
  
    System.out.println(  
        "\n===== QUIZ RESULTS ====="
```

```

        "Score: " +
        calculateScore()
    );

    System.out.println(
        "Total Questions: " +
        totalQuestions
    );
}
}

public class OnlineQuizManagementSystem {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        try {

            int questionId =
                sc.nextInt();
            sc.nextLine();

            String questionText =
                sc.nextLine();

            String correctAnswer =
                sc.nextLine();

```

```
int userId =  
    sc.nextInt();  
sc.nextLine();  
  
String userName =  
    sc.nextLine();  
  
String userAnswer =  
    sc.nextLine();  
  
if(!userAnswer.matches(  
    "[a-zA-Z0-9 ]+")) {  
  
    throw new IllegalArgumentException(  
        "Invalid Input Format"  
    );  
}  
  
Quiz q1 =  
    new Quiz(  
        questionId,  
        questionText,  
        correctAnswer,  
        userId,  
        userName,  
        userAnswer  
    );  
  
q1.displayQuiz();
```

```
}
```

```
catch(IllegalArgumentException e) {
```

```
    System.out.println(
        "Exception: " +
        e.getMessage()
    );
}
```

```
catch(ArithmeticException e) {
```

```
    System.out.println(
        "Score Exception: " +
        e.getMessage()
    );
}
```

```
catch(Exception e) {
```

```
    System.out.println(
        "Unexpected Error: " +
        e.getMessage()
    );
}
```

```
finally {
```

```
    System.out.println(
```

```

        "\nProgram Executed Successfully"

    );
}

    sc.close();
}
}

```

13.Customer Feedback and Rating System

- Create Customer, Feedback, Product classes.
- Interface: Reviewable, Package: com.feedback.system.
- Logic to accept ratings, calculate average rating per product.

Handle exceptions like **invalid rating values**, **duplicate feedback**, or **null reviews**.

(Use constructors, static, this, and final keywords wherever applicable)

Code :

```
package com.feedback.system;
```

```
import java.util.ArrayList;
```

```
import java.util.HashSet;
```

```
import java.util.Scanner;
```

```
interface Reviewable {
    void addReview(int rating, String review);
    double calculateAverageRating();
}

```

```
class Product {

    protected int productId;
    protected String productName;
}

```

```
static int totalReviews = 0;
```

```
final String companyName = "Online Store";
```

```
public Product(int productId,  
               String productName) {
```

```
    if(productName == null ||  
       productName.trim().isEmpty()) {
```

```
        throw new IllegalArgumentException(  
            "Product name cannot be null"  
        );  
    }
```

```
    this.productId = productId;  
    this.productName = productName;  
}
```

```
public void displayProduct() {
```

```
    System.out.println("Company Name: " +  
                       companyName);
```

```
    System.out.println("Product ID: " +  
                       productId);
```

```
    System.out.println("Product Name: " +
```

```
        productName);  
    }  
}
```

```
class Customer extends Product {
```

```
    protected int customerId;  
    protected String customerName;
```

```
    public Customer(int productId,  
                    String productName,  
                    int customerId,  
                    String customerName) {
```

```
        super(productId,  
                productName);
```

```
        if(customerName == null ||  
           customerName.trim().isEmpty()) {
```

```
            throw new IllegalArgumentException(  
                "Customer name cannot be null"  
            );  
        }
```

```
        this.customerId = customerId;  
        this.customerName = customerName;  
    }
```

```
public void displayCustomer() {

    System.out.println("Customer ID: " +
        customerId);

    System.out.println("Customer Name: " +
        customerName);
}
}

class Feedback extends Customer
    implements Reviewable {

    private ArrayList<Integer> ratings;

    private ArrayList<String> reviews;

    private static HashSet<Integer> feedbackIds =
        new HashSet<>();

    public Feedback(int productId,
        String productName,
        int customerId,
        String customerName) {

        super(productId,
            productName,
            customerId,
            customerName);
```

```

        ratings = new ArrayList<>();

        reviews = new ArrayList<>();
    }

    @Override
    public void addReview(int rating,
                          String review) {

        if(feedbackIds.contains(customerId)) {

            throw new IllegalArgumentException(
                "Duplicate Feedback"
            );
        }

        if(rating < 1 || rating > 5) {

            throw new IllegalArgumentException(
                "Invalid Rating Value"
            );
        }

        if(review == null ||
           review.trim().isEmpty()) {

            throw new IllegalArgumentException(
                "Null Review Not Allowed"
            );
        }
    }

```

```

        );
    }

    ratings.add(rating);

    reviews.add(review);

    feedbackIds.add(customerId);

    totalReviews++;

    System.out.println(
        "Feedback Added Successfully"
    );
}

@Override
public double calculateAverageRating() {

    int sum = 0;

    for(int r : ratings) {

        sum += r;
    }

    return (double) sum / ratings.size();
}

```

```
public void displayFeedback() {

    System.out.println(
        "\n===== FEEDBACK DETAILS ====="
    );

    displayProduct();

    displayCustomer();

    System.out.println(
        "Average Rating: " +
        calculateAverageRating()
    );

    System.out.println(
        "Total Reviews: " +
        totalReviews
    );
}
}
```

```
public class CustomerFeedbackSystem {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        try {
```

```
int productId =  
    sc.nextInt();  
sc.nextLine();
```

```
String productName =  
    sc.nextLine();
```

```
int customerId =  
    sc.nextInt();  
sc.nextLine();
```

```
String customerName =  
    sc.nextLine();
```

```
int rating =  
    sc.nextInt();  
sc.nextLine();
```

```
String review =  
    sc.nextLine();
```

```
Feedback f1 =  
    new Feedback(  
        productId,  
        productName,  
        customerId,  
        customerName  
    );
```

```
f1.addReview(  
    rating,  
    review  
);  
  
f1.displayFeedback();  
}  
  
catch(IllegalArgumentException e) {  
  
    System.out.println(  
        "Exception: " +  
        e.getMessage()  
    );  
}  
  
catch(Exception e) {  
  
    System.out.println(  
        "Unexpected Error: " +  
        e.getMessage()  
    );  
}  
  
finally {  
  
    System.out.println(  
        "\nProgram Executed Successfully"
```

```

        );
    }

    sc.close();
}
}

```

14. Food Delivery App Simulation

- **Classes:** Restaurant, Order, DeliveryPerson. Use composition.
- **Interface:** Deliverable, **Package:** com.fooddelivery.app.
- Logic for placing, tracking, and delivering orders.

Manage exceptions like **order delays**, **unavailable delivery personnel**, or **invalid order format**.

(Use constructors, static, this, and final keywords wherever applicable)

Code :

```
package com.fooddelivery.app;
```

```
import java.util.Scanner;
```

```
interface Deliverable {
    void placeOrder();
    void trackOrder();
    void deliverOrder();
}

```

```
class Restaurant {
```

```
    protected int restaurantId;
    protected String restaurantName;
```

```
    static int totalOrders = 0;
```

```
final String appName = "Food Delivery App";
```

```
public Restaurant(int restaurantId,  
                  String restaurantName) {
```

```
    if(restaurantName == null ||  
        restaurantName.trim().isEmpty()) {
```

```
        throw new IllegalArgumentException(  
            "Invalid Restaurant Name"  
        );  
    }
```

```
    this.restaurantId = restaurantId;  
    this.restaurantName = restaurantName;  
}
```

```
public void displayRestaurant() {
```

```
    System.out.println("App Name: " +  
        appName);
```

```
    System.out.println("Restaurant ID: " +  
        restaurantId);
```

```
    System.out.println("Restaurant Name: " +  
        restaurantName);  
}
```

```
}
```

```
class Order {
```

```
    private int orderId;
```

```
    private String foodItem;
```

```
    private double price;
```

```
    public Order(int orderId,  
                 String foodItem,  
                 double price) {
```

```
        if(foodItem == null ||  
           foodItem.trim().isEmpty()) {
```

```
            throw new IllegalArgumentException(  
                "Invalid Order Format"  
            );  
        }
```

```
        if(price <= 0) {
```

```
            throw new IllegalArgumentException(  
                "Invalid Food Price"  
            );  
        }
```

```
        this.orderId = orderId;
```

```
        this.foodItem = foodItem;
```

```
    this.price = price;
}
```

```
public void displayOrder() {
```

```
    System.out.println("Order ID: " +
        orderId);
```

```
    System.out.println("Food Item: " +
        foodItem);
```

```
    System.out.println("Price: " +
        price);
```

```
}
```

```
}
```

```
class DeliveryPerson {
```

```
    private int deliveryId;
```

```
    private String deliveryName;
```

```
    private boolean available;
```

```
    public DeliveryPerson(int deliveryId,
        String deliveryName,
        boolean available) {
```

```
        if(deliveryName == null ||
            deliveryName.trim().isEmpty()) {
```

```
        throw new IllegalArgumentException(  
            "Invalid Delivery Person"  
        );  
    }
```

```
    this.deliveryId = deliveryId;  
    this.deliveryName = deliveryName;  
    this.available = available;  
}
```

```
public boolean isAvailable() {  
    return available;  
}
```

```
public void displayDeliveryPerson() {
```

```
    System.out.println("Delivery ID: " +  
        deliveryId);
```

```
    System.out.println("Delivery Name: " +  
        deliveryName);
```

```
    System.out.println("Availability: " +  
        available);
```

```
    }  
}
```

```
class FoodDelivery  
    implements Deliverable {
```

```
private Restaurant restaurant;
```

```
private Order order;
```

```
private DeliveryPerson deliveryPerson;
```

```
public FoodDelivery(Restaurant restaurant,  
                    Order order,  
                    DeliveryPerson deliveryPerson) {
```

```
    if(restaurant == null ||  
        order == null ||  
        deliveryPerson == null) {
```

```
        throw new IllegalArgumentException(  
            "Null Object Reference"  
        );  
    }
```

```
    this.restaurant = restaurant;  
    this.order = order;  
    this.deliveryPerson = deliveryPerson;  
}
```

```
@Override
```

```
public void placeOrder() {
```

```
    Restaurant.totalOrders++;
```

```
        System.out.println(  
            "Order Placed Successfully"  
        );  
    }
```

```
@Override  
public void trackOrder() {
```

```
    System.out.println(  
        "Order is being prepared"  
    );  
}
```

```
@Override  
public void deliverOrder() {
```

```
    if(!deliveryPerson.isAvailable()) {  
  
        throw new IllegalArgumentException(  
            "Delivery Person Unavailable"  
        );  
    }
```

```
    System.out.println(  
        "Order Delivered Successfully"  
    );  
}
```

```
public void orderDelay(int minutes) {

    if(minutes > 60) {

        throw new IllegalArgumentException(
            "Order Delay Detected"
        );
    }
}

public void displayDetails() {

    System.out.println(
        "\n===== DELIVERY DETAILS ====="
    );

    restaurant.displayRestaurant();

    order.displayOrder();

    deliveryPerson.displayDeliveryPerson();

    System.out.println(
        "Total Orders: " +
        Restaurant.totalOrders
    );
}
}
```

```
public class FoodDeliveryAppSimulation {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        try {
```

```
            int restaurantId =
```

```
                sc.nextInt();
```

```
            sc.nextLine();
```

```
            String restaurantName =
```

```
                sc.nextLine();
```

```
            Restaurant r1 =
```

```
                new Restaurant(
```

```
                    restaurantId,
```

```
                    restaurantName
```

```
                );
```

```
            int orderId =
```

```
                sc.nextInt();
```

```
            sc.nextLine();
```

```
            String foodItem =
```

```
                sc.nextLine();
```

```
            double price =
```

```
sc.nextDouble();
```

```
Order o1 =
```

```
    new Order(  
        orderId,  
        foodItem,  
        price  
    );
```

```
int deliveryId =
```

```
    sc.nextInt();  
sc.nextLine();
```

```
String deliveryName =
```

```
    sc.nextLine();
```

```
boolean available =
```

```
    sc.nextBoolean();
```

```
DeliveryPerson d1 =
```

```
    new DeliveryPerson(  
        deliveryId,  
        deliveryName,  
        available  
    );
```

```
int delayMinutes =
```

```
    sc.nextInt();
```

```
FoodDelivery fd =  
    new FoodDelivery(  
        r1,  
        o1,  
        d1  
    );  
  
fd.placeOrder();  
  
fd.trackOrder();  
  
fd.orderDelay(delayMinutes);  
  
fd.deliverOrder();  
  
fd.displayDetails();  
}  
  
catch(IllegalArgumentException e) {  
  
    System.out.println(  
        "Exception: " +  
        e.getMessage()  
    );  
}  
  
catch(Exception e) {  
  
    System.out.println(
```

```

        "Unexpected Error: " +
        e.getMessage()
    );
}

finally {

    System.out.println(
        "\nProgram Executed Successfully"
    );
}

sc.close();
}
}

```

15.Parcel Tracking System

- Use `Parcel`, `Location`, and `Courier` classes.
- Package: `com.parcel.tracking`, Interface: `Trackable`.
- Logic for updating status and delivery time prediction.

Handle **missing location updates**, **invalid tracking number**.

(Use constructors, static, this, and final keywords wherever applicable)

Code :

```

package com.parcel.tracking;

import java.util.Scanner;

interface Trackable {
    void updateStatus(String status);
    void predictDeliveryTime(int days);
}

```

```
}
```

```
class Parcel {
```

```
    protected String trackingNumber;
```

```
    protected String senderName;
```

```
    protected String receiverName;
```

```
    static int totalParcels = 0;
```

```
    final String companyName = "Fast Courier Service";
```

```
    public Parcel(String trackingNumber,
```

```
                  String senderName,
```

```
                  String receiverName) {
```

```
        if(trackingNumber == null ||
```

```
           trackingNumber.trim().isEmpty()) {
```

```
            throw new IllegalArgumentException(
```

```
                "Invalid Tracking Number"
```

```
            );
```

```
        }
```

```
        if(senderName == null ||
```

```
           senderName.trim().isEmpty() ||
```

```
           receiverName == null ||
```

```
           receiverName.trim().isEmpty()) {
```

```
        throw new IllegalArgumentException(
            "Sender or Receiver Name Missing"
        );
    }

    this.trackingNumber = trackingNumber;
    this.senderName = senderName;
    this.receiverName = receiverName;

    totalParcels++;
}

public void displayParcel() {

    System.out.println("Company Name: " +
        companyName);

    System.out.println("Tracking Number: " +
        trackingNumber);

    System.out.println("Sender Name: " +
        senderName);

    System.out.println("Receiver Name: " +
        receiverName);
}
}

class Location extends Parcel {
```

```
protected String currentLocation;
```

```
public Location(String trackingNumber,  
                String senderName,  
                String receiverName,  
                String currentLocation) {
```

```
    super(trackingNumber,  
          senderName,  
          receiverName);
```

```
    if(currentLocation == null ||  
        currentLocation.trim().isEmpty()) {  
  
        throw new IllegalArgumentException(  
            "Missing Location Update"  
        );  
    }
```

```
    this.currentLocation =  
        currentLocation;  
}
```

```
public void displayLocation() {
```

```
    System.out.println("Current Location: " +  
        currentLocation);  
}
```

```
}
```

```
class Courier extends Location
```

```
    implements Trackable {
```

```
    private String status;
```

```
    private int deliveryDays;
```

```
    public Courier(String trackingNumber,
```

```
        String senderName,
```

```
        String receiverName,
```

```
        String currentLocation) {
```

```
        super(trackingNumber,
```

```
            senderName,
```

```
            receiverName,
```

```
            currentLocation);
```

```
    }
```

```
    @Override
```

```
    public void updateStatus(String status) {
```

```
        if(status == null ||
```

```
            status.trim().isEmpty()) {
```

```
            throw new IllegalArgumentException(
```

```
                "Invalid Parcel Status"
```

```
            );
```

```
}
```

```
this.status = status;
```

```
System.out.println(  
    "Parcel Status Updated"
```

```
);
```

```
}
```

```
@Override
```

```
public void predictDeliveryTime(int days) {
```

```
    if(days <= 0) {
```

```
        throw new IllegalArgumentException(  
            "Invalid Delivery Days"
```

```
        );
```

```
    }
```

```
    this.deliveryDays = days;
```

```
System.out.println(  
    "Estimated Delivery Time: " +  
    deliveryDays + " days"
```

```
);
```

```
}
```

```
public void displayCourier() {
```

```
        System.out.println(
            "\n===== PARCEL TRACKING DETAILS ====="
        );

        displayParcel();

        displayLocation();

        System.out.println("Parcel Status: " +
            status);

        System.out.println("Delivery In: " +
            deliveryDays + " days");

        System.out.println(
            "Total Parcels: " +
            totalParcels
        );
    }
}
```

```
public class ParcelTrackingSystem {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        try {
```

```
String trackingNumber =  
    sc.nextLine();
```

```
String senderName =  
    sc.nextLine();
```

```
String receiverName =  
    sc.nextLine();
```

```
String currentLocation =  
    sc.nextLine();
```

```
String status =  
    sc.nextLine();
```

```
int deliveryDays =  
    sc.nextInt();
```

```
Courier c1 =  
    new Courier(  
        trackingNumber,  
        senderName,  
        receiverName,  
        currentLocation  
    );
```

```
c1.updateStatus(status);
```

```
c1.predictDeliveryTime(
```

```
        deliveryDays
    );

    c1.displayCourier();
}

catch(IllegalArgumentException e) {

    System.out.println(
        "Exception: " +
        e.getMessage()
    );
}

catch(Exception e) {

    System.out.println(
        "Unexpected Error: " +
        e.getMessage()
    );
}

finally {

    System.out.println(
        "\nProgram Executed Successfully"
    );
}
```

```
        sc.close();
    }
}
```

16. Electricity Bill Generator

- **Classes:** `Customer`, `Bill`, `Meter`. Use inheritance.
- **Interface:** `Billable`, **Package:** `com.utility.billing`.
- Logic for unit-based billing and applying slabs.

Catch **invalid meter readings**, **negative units**, or **slab mismatch**.

(Use constructors, static, this, and final keywords wherever applicable)

Code :

```
package com.utility.billing;
```

```
import java.util.Scanner;
```

```
interface Billable {
    double calculateBill();
}
```

```
class Customer {
```

```
    protected int customerId;
```

```
    protected String customerName;
```

```
    static int totalBills = 0;
```

```
    final String companyName = "State Electricity Board";
```

```
    public Customer(int customerId,
                    String customerName) {
```

```

        if(customerName == null ||
            customerName.trim().isEmpty()) {

            throw new IllegalArgumentException(
                "Invalid Customer Name"
            );
        }

        this.customerId = customerId;
        this.customerName = customerName;
    }

    public void displayCustomer() {

        System.out.println("Company Name: " +
            companyName);

        System.out.println("Customer ID: " +
            customerId);

        System.out.println("Customer Name: " +
            customerName);
    }
}

class Meter extends Customer {

    protected int previousReading;

```

```
protected int currentReading;
```

```
protected int unitsConsumed;
```

```
public Meter(int customerId,  
             String customerName,  
             int previousReading,  
             int currentReading) {
```

```
    super(customerId,  
          customerName);
```

```
    if(previousReading < 0 ||  
       currentReading < 0) {
```

```
        throw new IllegalArgumentException(  
            "Meter readings cannot be negative"  
        );  
    }
```

```
    if(currentReading < previousReading) {
```

```
        throw new IllegalArgumentException(  
            "Invalid Meter Reading"  
        );  
    }
```

```
    this.previousReading =  
        previousReading;
```

```
this.currentReading =  
    currentReading;  
  
this.unitsConsumed =  
    currentReading - previousReading;  
}  
  
public void displayMeter() {  
  
    System.out.println("Previous Reading: " +  
        previousReading);  
  
    System.out.println("Current Reading: " +  
        currentReading);  
  
    System.out.println("Units Consumed: " +  
        unitsConsumed);  
}  
}
```

```
class Bill extends Meter  
    implements Billable {  
  
    private double totalAmount;  
  
    public Bill(int customerId,  
        String customerName,  
        int previousReading,  
        int currentReading) {
```

```
super(customerId,  
        customerName,  
        previousReading,  
        currentReading);  
}
```

```
@Override
```

```
public double calculateBill() {
```

```
    if(unitsConsumed < 0) {
```

```
        throw new IllegalArgumentException(  
            "Negative Units Not Allowed"  
        );  
    }
```

```
    if(unitsConsumed <= 100) {
```

```
        totalAmount =  
            unitsConsumed * 5;  
    }
```

```
    else if(unitsConsumed <= 300) {
```

```
        totalAmount =  
            (100 * 5) +  
            ((unitsConsumed - 100) * 7);  
    }
```

```
else if(unitsConsumed > 300) {

    totalAmount =

        (100 * 5) +

        (200 * 7) +

        ((unitsConsumed - 300) * 10);

}

else {

    throw new IllegalArgumentException(

        "Slab Mismatch Error"

    );

}

totalBills++;

return totalAmount;

}

public void displayBill() {

    System.out.println(

        "\n===== ELECTRICITY BILL ====="

    );

    displayCustomer();

}
```

```
displayMeter();
```

```
System.out.println("Total Bill Amount: " +  
    calculateBill());
```

```
System.out.println("Total Bills Generated: " +  
    totalBills);
```

```
}
```

```
}
```

```
public class ElectricityBillGenerator {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        try {
```

```
            int customerId =
```

```
                sc.nextInt();
```

```
            sc.nextLine();
```

```
            String customerName =
```

```
                sc.nextLine();
```

```
            int previousReading =
```

```
                sc.nextInt();
```

```
            int currentReading =
```

```
sc.nextInt();
```

```
Bill b1 =  
    new Bill(  
        customerId,  
        customerName,  
        previousReading,  
        currentReading  
    );  
  
b1.displayBill();  
}
```

```
catch(IllegalArgumentException e) {
```

```
    System.out.println(  
        "Exception: " +  
        e.getMessage()  
    );  
}
```

```
catch(Exception e) {
```

```
    System.out.println(  
        "Unexpected Error: " +  
        e.getMessage()  
    );  
}
```

```

        finally {

            System.out.println(

                "\nProgram Executed Successfully"

            );

        }

        sc.close();

    }

}

```

17.Event Management System

- Create classes `Event`, `Venue`, `Attendee`.
- Package: `com.events.manager`, Interface: `Schedulable`.
- Logic for managing bookings and available slots.

Handle **event overbooking**, **double bookings**, **invalid dates**.

(Use constructors, static, this, and final keywords wherever applicable)

Code :

```
package com.events.manager;
```

```
import java.util.ArrayList;
```

```
import java.util.Scanner;
```

```
interface Schedulable {

    void bookSlot();

}
```

```
class Event {

    protected int eventId;
```

```
protected String eventName;  
protected String eventDate;  
  
static int totalBookings = 0;  
  
final String organizerName =  
    "Event Management Company";  
  
public Event(int eventId,  
             String eventName,  
             String eventDate) {  
  
    if(eventName == null ||  
        eventName.trim().isEmpty()) {  
  
        throw new IllegalArgumentException(  
            "Invalid Event Name"  
        );  
    }  
  
    if(eventDate == null ||  
        eventDate.trim().isEmpty()) {  
  
        throw new IllegalArgumentException(  
            "Invalid Event Date"  
        );  
    }  
  
    this.eventId = eventId;
```

```
        this.eventName = eventName;
        this.eventDate = eventDate;
    }

    public void displayEvent() {

        System.out.println("Organizer: " +
            organizerName);

        System.out.println("Event ID: " +
            eventId);

        System.out.println("Event Name: " +
            eventName);

        System.out.println("Event Date: " +
            eventDate);
    }
}
```

```
class Venue extends Event {

    protected String venueName;
    protected int totalSlots;
    protected int availableSlots;

    public Venue(int eventId,
        String eventName,
        String eventDate,
```

```
        String venueName,  
        int totalSlots) {  
  
    super(eventId,  
        eventName,  
        eventDate);  
  
    if(venueName == null ||  
        venueName.trim().isEmpty()) {  
  
        throw new IllegalArgumentException(  
            "Invalid Venue Name"  
        );  
    }  
  
    if(totalSlots <= 0) {  
  
        throw new IllegalArgumentException(  
            "Invalid Slot Count"  
        );  
    }  
  
    this.venueName = venueName;  
    this.totalSlots = totalSlots;  
    this.availableSlots = totalSlots;  
}  
  
public void displayVenue() {
```

```
        System.out.println("Venue Name: " +
            venueName);

        System.out.println("Available Slots: " +
            availableSlots);
    }
}
```

```
class Attendee extends Venue
    implements Schedulable {

    private int attendeeId;
    private String attendeeName;

    private static ArrayList<Integer>
        bookedAttendees =
            new ArrayList<>();

    public Attendee(int eventId,
        String eventName,
        String eventDate,
        String venueName,
        int totalSlots,
        int attendeeId,
        String attendeeName) {

        super(eventId,
            eventName,
            eventDate,
```

```
venueName,  
totalSlots);
```

```
if(attendeeName == null ||  
attendeeName.trim().isEmpty()) {  
  
    throw new IllegalArgumentException(  
        "Invalid Attendee Name"  
    );  
}
```

```
this.attendeeld = attendeeId;  
this.attendeeName = attendeeName;  
}
```

```
@Override
```

```
public void bookSlot() {
```

```
    if(bookedAttendees.contains(  
        attendeeId)) {  
  
        throw new IllegalArgumentException(  
            "Double Booking Detected"  
        );  
    }
```

```
    if(availableSlots <= 0) {  
  
        throw new IllegalArgumentException(  

```

```

        "Event Overbooked"

    );
}

bookedAttendees.add(attendeeId);

availableSlots--;

totalBookings++;

System.out.println(
    "Event Slot Booked Successfully"
);
}

public void validateDate() {

    if(!eventDate.matches(
        "\\d{2}-\\d{2}-\\d{4}")) {

        throw new IllegalArgumentException(
            "Invalid Date Format"
        );
    }
}

```

```

public void displayAttendee() {

```

```

    System.out.println(

```

```

        "\n===== EVENT DETAILS ====="
    );

    displayEvent();

    displayVenue();

    System.out.println("Attendee ID: " +
        attendeeId);

    System.out.println("Attendee Name: " +
        attendeeName);

    System.out.println("Total Bookings: " +
        totalBookings);
    }
}

```

```

public class EventManagementSystem {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        try {

            int eventId =

                sc.nextInt();

            sc.nextLine();

```

```
String eventName =  
    sc.nextLine();
```

```
String eventDate =  
    sc.nextLine();
```

```
String venueName =  
    sc.nextLine();
```

```
int totalSlots =  
    sc.nextInt();
```

```
int attendeeId =  
    sc.nextInt();  
sc.nextLine();
```

```
String attendeeName =  
    sc.nextLine();
```

```
Attendee a1 =  
    new Attendee(  
        eventId,  
        eventName,  
        eventDate,  
        venueName,  
        totalSlots,  
        attendeeId,  
        attendeeName
```

```
        );

        a1.validateDate();

        a1.bookSlot();

        a1.displayAttendee();
    }

    catch(IllegalArgumentException e) {

        System.out.println(
            "Exception: " +
            e.getMessage()
        );
    }

    catch(Exception e) {

        System.out.println(
            "Unexpected Error: " +
            e.getMessage()
        );
    }

    finally {

        System.out.println(
            "\nProgram Executed Successfully"
```

```

        );
    }

    sc.close();
}
}

```

18. Hotel Room Booking System

- Define Room, Customer, Booking classes.
- Interface: Bookable, Package: com.hotel.booking.
- Logic for room type selection, availability, and pricing.

Handle **no rooms available, invalid customer info, payment failures.**

(Use constructors, static, this, and final keywords wherever applicable)

Code :

```
package com.hotel.booking;
```

```
import java.util.ArrayList;
```

```
import java.util.Scanner;
```

```
interface Bookable {
    void bookRoom();
}

```

```
class Room {

    protected int roomNumber;
    protected String roomType;
    protected double roomPrice;
    protected boolean isAvailable;
}

```

```
static int totalBookings = 0;
```

```
final String hotelName = "Grand Palace Hotel";
```

```
public Room(int roomNumber,  
            String roomType,  
            double roomPrice,  
            boolean isAvailable) {
```

```
    if(roomType == null ||  
        roomType.trim().isEmpty()) {
```

```
        throw new IllegalArgumentException(  
            "Invalid Room Type"  
        );  
    }
```

```
    if(roomPrice <= 0) {
```

```
        throw new IllegalArgumentException(  
            "Invalid Room Price"  
        );  
    }
```

```
    this.roomNumber = roomNumber;  
    this.roomType = roomType;  
    this.roomPrice = roomPrice;  
    this.isAvailable = isAvailable;  
}
```

```
public void displayRoom() {  
  
    System.out.println("Hotel Name: " +  
        hotelName);  
  
    System.out.println("Room Number: " +  
        roomNumber);  
  
    System.out.println("Room Type: " +  
        roomType);  
  
    System.out.println("Room Price: " +  
        roomPrice);  
  
    System.out.println("Availability: " +  
        isAvailable);  
}  
}
```

```
class Customer extends Room {  
  
    protected int customerId;  
    protected String customerName;  
  
    public Customer(int roomNumber,  
        String roomType,  
        double roomPrice,  
        boolean isAvailable,
```

```
        int customerId,
        String customerName) {

    super(roomNumber,
        roomType,
        roomPrice,
        isAvailable);

    if(customerName == null ||
        customerName.trim().isEmpty()) {

        throw new IllegalArgumentException(
            "Invalid Customer Information"
        );
    }

    this.customerId = customerId;
    this.customerName = customerName;
}

public void displayCustomer() {

    System.out.println("Customer ID: " +
        customerId);

    System.out.println("Customer Name: " +
        customerName);
}
}
```

```
class Booking extends Customer
    implements Bookable {

    private boolean paymentStatus;

    private static ArrayList<Integer>
        bookedRooms =
            new ArrayList<>();

    public Booking(int roomNumber,
        String roomType,
        double roomPrice,
        boolean isAvailable,
        int customerId,
        String customerName,
        boolean paymentStatus) {

        super(roomNumber,
            roomType,
            roomPrice,
            isAvailable,
            customerId,
            customerName);

        this.paymentStatus =
            paymentStatus;
    }
}
```

@Override

public void bookRoom() {

if(bookedRooms.contains(
 roomNumber)) {

throw new IllegalArgumentException(
 "Room Already Booked"
);
 }

if(!isAvailable) {

throw new IllegalArgumentException(
 "No Rooms Available"
);
 }

if(!paymentStatus) {

throw new IllegalArgumentException(
 "Payment Failure"
);
 }

bookedRooms.add(roomNumber);

isAvailable = false;

```
totalBookings++;
```

```
System.out.println(  
    "Room Booked Successfully"  
);  
}
```

```
public void displayBooking() {
```

```
System.out.println(  
    "\n===== HOTEL BOOKING DETAILS ====="  
);
```

```
displayRoom();
```

```
displayCustomer();
```

```
System.out.println("Payment Status: " +  
    paymentStatus);
```

```
System.out.println("Total Bookings: " +  
    totalBookings);  
}  
}
```

```
public class HotelRoomBookingSystem {
```

```
public static void main(String[] args) {
```

```
Scanner sc = new Scanner(System.in);
```

```
try {
```

```
    int roomNumber =
```

```
        sc.nextInt();
```

```
    sc.nextLine();
```

```
    String roomType =
```

```
        sc.nextLine();
```

```
    double roomPrice =
```

```
        sc.nextDouble();
```

```
    boolean isAvailable =
```

```
        sc.nextBoolean();
```

```
    int customerId =
```

```
        sc.nextInt();
```

```
    sc.nextLine();
```

```
    String customerName =
```

```
        sc.nextLine();
```

```
    boolean paymentStatus =
```

```
        sc.nextBoolean();
```

```
    Booking b1 =
```

```
        new Booking(
```

```
        roomNumber,  
        roomType,  
        roomPrice,  
        isAvailable,  
        customerId,  
        customerName,  
        paymentStatus  
    );  
  
    b1.bookRoom();  
  
    b1.displayBooking();  
}  
  
catch(IllegalArgumentException e) {  
  
    System.out.println(  
        "Exception: " +  
        e.getMessage()  
    );  
}  
  
catch(Exception e) {  
  
    System.out.println(  
        "Unexpected Error: " +  
        e.getMessage()  
    );  
}
```

```

        finally {

            System.out.println(

                "\nProgram Executed Successfully"

            );

        }

        sc.close();

    }

}

```

19.Loan Eligibility Checker

- Create classes `Loan`, `Applicant`, `CreditScore`.
- Package: `com.finance.loans`, Interface: `Eligible`.
- Logic to check eligibility based on age, income, credit score.

Catch **incomplete applicant data, invalid scores, or rule violations**.

(Use constructors, static, this, and final keywords wherever applicable)

Code :

```
package com.finance.loans;
```

```
import java.util.Scanner;
```

```
interface Eligible {

    boolean checkEligibility();

}

```

```
class Loan {

    protected int loanId;

```

```
protected String loanType;
protected double loanAmount;

static int totalApplications = 0;

final String bankName = "National Finance Bank";

public Loan(int loanId,
            String loanType,
            double loanAmount) {

    if(loanType == null ||
        loanType.trim().isEmpty()) {

        throw new IllegalArgumentException(
            "Invalid Loan Type"
        );
    }

    if(loanAmount <= 0) {

        throw new IllegalArgumentException(
            "Invalid Loan Amount"
        );
    }

    this.loanId = loanId;
    this.loanType = loanType;
    this.loanAmount = loanAmount;
```

```
        totalApplications++;
    }

    public void displayLoan() {

        System.out.println("Bank Name: " +
            bankName);

        System.out.println("Loan ID: " +
            loanId);

        System.out.println("Loan Type: " +
            loanType);

        System.out.println("Loan Amount: " +
            loanAmount);
    }
}
```

```
class Applicant extends Loan {

    protected int applicantId;
    protected String applicantName;
    protected int age;
    protected double income;

    public Applicant(int loanId,
        String loanType,
```

```
        double loanAmount,
        int applicantId,
        String applicantName,
        int age,
        double income) {

    super(loanId,
        loanType,
        loanAmount);

    if(applicantName == null ||
        applicantName.trim().isEmpty()) {

        throw new IllegalArgumentException(
            "Incomplete Applicant Data"
        );
    }

    if(age < 18 || age > 65) {

        throw new IllegalArgumentException(
            "Invalid Applicant Age"
        );
    }

    if(income <= 0) {

        throw new IllegalArgumentException(
            "Invalid Income"
```

```
);  
}
```

```
    this.applicantId = applicantId;  
    this.applicantName = applicantName;  
    this.age = age;  
    this.income = income;  
}
```

```
public void displayApplicant() {
```

```
    System.out.println("Applicant ID: " +  
        applicantId);
```

```
    System.out.println("Applicant Name: " +  
        applicantName);
```

```
    System.out.println("Age: " + age);
```

```
    System.out.println("Income: " +  
        income);
```

```
    }  
}
```

```
class CreditScore extends Applicant  
    implements Eligible {  
  
    private int creditScore;
```

```
public CreditScore(int loanId,  
    String loanType,  
    double loanAmount,  
    int applicantId,  
    String applicantName,  
    int age,  
    double income,  
    int creditScore) {
```

```
    super(loanId,  
        loanType,  
        loanAmount,  
        applicantId,  
        applicantName,  
        age,  
        income);
```

```
    if(creditScore < 300 ||  
        creditScore > 900) {
```

```
        throw new IllegalArgumentException(  
            "Invalid Credit Score"  
        );  
    }
```

```
    this.creditScore = creditScore;  
}
```

@Override

```
public boolean checkEligibility() {
```

```
    if(age < 21 ||
```

```
        income < 25000 ||
```

```
        creditScore < 650) {
```

```
        return false;
```

```
    }
```

```
    return true;
```

```
}
```

```
public void displayEligibility() {
```

```
    System.out.println(
```

```
        "\n===== LOAN ELIGIBILITY DETAILS ====="
```

```
    );
```

```
    displayLoan();
```

```
    displayApplicant();
```

```
    System.out.println("Credit Score: " +
```

```
        creditScore);
```

```
    System.out.println("Eligible: " +
```

```
        checkEligibility());
```

```
    System.out.println(
```

```
        "Total Applications: " +  
        totalApplications  
    );  
}  
}
```

```
public class LoanEligibilityChecker {  
  
    public static void main(String[] args) {  
  
        Scanner sc = new Scanner(System.in);  
  
        try {  
  
            int loanId =  
                sc.nextInt();  
            sc.nextLine();  
  
            String loanType =  
                sc.nextLine();  
  
            double loanAmount =  
                sc.nextDouble();  
  
            int applicantId =  
                sc.nextInt();  
            sc.nextLine();  
  
            String applicantName =
```

```
        sc.nextLine();

    int age =

        sc.nextInt();

    double income =

        sc.nextDouble();

    int creditScore =

        sc.nextInt();

    CreditScore c1 =

        new CreditScore(

            loanId,

            loanType,

            loanAmount,

            applicantId,

            applicantName,

            age,

            income,

            creditScore

        );

    c1.displayEligibility();
}

catch(IllegalArgumentException e) {

    System.out.println(
```

```

        "Exception: " +
        e.getMessage()
    );
}

catch(Exception e) {

    System.out.println(
        "Unexpected Error: " +
        e.getMessage()
    );
}

finally {

    System.out.println(
        "\nProgram Executed Successfully"
    );
}

sc.close();
}
}

```

20.Weather Forecasting Application

- Classes: WeatherData, City, Forecast.
- Interface: Forecastable, Package: com.weather.app.
- Logic to calculate temperature average, humidity trends using data arrays.

Handle **missing data entries, invalid temperature formats, null references.**

(Use constructors, static, this, and final keywords wherever applicable)

Code :

```
package com.weather.app;
```

```
import java.util.Scanner;
```

```
interface Forecastable {  
  
    double calculateAverageTemperature();  
  
    void showHumidityTrend();  
  
}
```

```
class WeatherData {  
  
    protected double[] temperatures;  
    protected int[] humidityLevels;  
  
    static int totalForecasts = 0;  
  
    final String appName = "Weather Forecast App";  
  
    public WeatherData(double[] temperatures,  
                        int[] humidityLevels) {  
  
        if(temperatures == null ||
```

```
        humidityLevels == null) {

        throw new IllegalArgumentException(

            "Null Data Reference"

        );

    }

    if(temperatures.length == 0 ||

        humidityLevels.length == 0) {

        throw new IllegalArgumentException(

            "Missing Data Entries"

        );

    }

    this.temperatures = temperatures;

    this.humidityLevels = humidityLevels;

    totalForecasts++;

}

public void displayWeatherData() {

    System.out.println("Application Name: " +
```

```
    appName);
```

```
System.out.println("Temperature Data:");
```

```
for(double t : temperatures) {
```

```
    System.out.print(t + " ");
```

```
}
```

```
System.out.println("\nHumidity Data:");
```

```
for(int h : humidityLevels) {
```

```
    System.out.print(h + " ");
```

```
}
```

```
System.out.println();
```

```
}
```

```
}
```

```
class City extends WeatherData {
```

```
    protected int cityId;
```

```
    protected String cityName;
```

```
public City(int cityId,
            String cityName,
            double[] temperatures,
            int[] humidityLevels) {

    super(temperatures,
          humidityLevels);

    if(cityName == null ||
       cityName.trim().isEmpty()) {

        throw new IllegalArgumentException(
            "Invalid City Name"
        );
    }

    this.cityId = cityId;
    this.cityName = cityName;
}

public void displayCity() {

    System.out.println("City ID: " +
```

```
        cityId);

        System.out.println("City Name: " +
            cityName);
    }
}
```

```
class Forecast extends City
    implements Forecastable {

    public Forecast(int cityId,
        String cityName,
        double[] temperatures,
        int[] humidityLevels) {

        super(cityId,
            cityName,
            temperatures,
            humidityLevels);
    }
}
```

```
@Override
public double calculateAverageTemperature() {
```

```
double sum = 0;

for(double t : temperatures) {

    if(t < -100 || t > 100) {

        throw new IllegalArgumentException(

            "Invalid Temperature Format"

        );

    }

    sum += t;

}

return sum / temperatures.length;

}

@Override

public void showHumidityTrend() {

    System.out.println(

        "\nHumidity Trend:"

    );

}
```

```
for(int h : humidityLevels) {

    if(h < 0 || h > 100) {

        throw new IllegalArgumentException(

            "Invalid Humidity Value"

        );

    }

    if(h > 70) {

        System.out.println(

            h + "% -> High Humidity"

        );

    }

    else {

        System.out.println(

            h + "% -> Normal Humidity"

        );

    }

}

}
```

```
public void displayForecast() {

    System.out.println(

        "\n===== WEATHER FORECAST DETAILS ====="

    );

    displayCity();

    displayWeatherData();

    System.out.println(

        "Average Temperature: " +

        calculateAverageTemperature()

    );

    showHumidityTrend();

    System.out.println(

        "\nTotal Forecast Records: " +

        totalForecasts

    );

}

}
```

```
public class WeatherForecastingApplication {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        try {
```

```
            int cityId =
```

```
                sc.nextInt();
```

```
            sc.nextLine();
```

```
            String cityName =
```

```
                sc.nextLine();
```

```
            int size =
```

```
                sc.nextInt();
```

```
            double[] temperatures =
```

```
                new double[size];
```

```
            int[] humidityLevels =
```

```
                new int[size];
```

```
for(int i = 0; i < size; i++) {
```

```
    temperatures[i] =
```

```
        sc.nextDouble();
```

```
}
```

```
for(int i = 0; i < size; i++) {
```

```
    humidityLevels[i] =
```

```
        sc.nextInt();
```

```
}
```

```
Forecast f1 =
```

```
    new Forecast(
```

```
        cityId,
```

```
        cityName,
```

```
        temperatures,
```

```
        humidityLevels
```

```
    );
```

```
f1.displayForecast();
```

```
}
```

```
catch(IllegalArgumentException e) {
```

```
    System.out.println(
```

```
        "Exception: " +
```

```
        e.getMessage()
```

```
    );
```

```
}
```

```
catch(Exception e) {
```

```
    System.out.println(
```

```
        "Unexpected Error: " +
```

```
        e.getMessage()
```

```
    );
```

```
}
```

```
finally {
```

```
    System.out.println(
```

```
        "\nProgram Executed Successfully"
```

```
    );
```

```
}
```

```
sc.close();
```

}

}